

```

9
10  const body = razorpay_order_id + "|" + razorpay_payment_id;
11  const expectedSignature = crypto
12    .createHmac('sha256', process.env.RAZORPAY_KEY_SECRET!)
13    .update(body.toString())
14    .digest('hex');
15
16  if (expectedSignature !== razorpay_signature) {
17    return NextResponse.json({ error: 'Invalid' }, { status:
18      400 });
19  }
20
21  // Use Firebase Admin for reliable server-side persistence
22  const { db } = await import('@/lib/firebase-server');
23  const { FieldValue } = await import('firebase-admin/firestore
24    ');
25
26  const orderRef = await db.collection('orders').add({
27    listingId, buyerId, sellerId, price,
28    status: 'Paid', createdAt: FieldValue.serverTimestamp(),
29  });
30
31  await db.collection('listings').doc(listingId).update({
32    paymentStatus: 'PAID', orderId: orderRef.id,
33    updatedAt: FieldValue.serverTimestamp(),
34  });
35
36  return NextResponse.json({ ok: true, orderId: orderRef.id });
37 } catch (err) {
38   return NextResponse.json({ error: 'Error' }, { status: 500 })
    ;
  }
}

```

Listing 8.12: Razorpay Payment Verification (api/razorpay/verify/route.ts)

This verification step is critical for security as it prevents malicious users from forging payment success responses. By performing this check on the server using the RAZORPAY_KEY_SECRET and then updating the database using the Firebase Admin SDK, we ensure that the transaction is legitimate and the listing status is accurately reflected.